Хранилища, очереди и брокеры сообщений

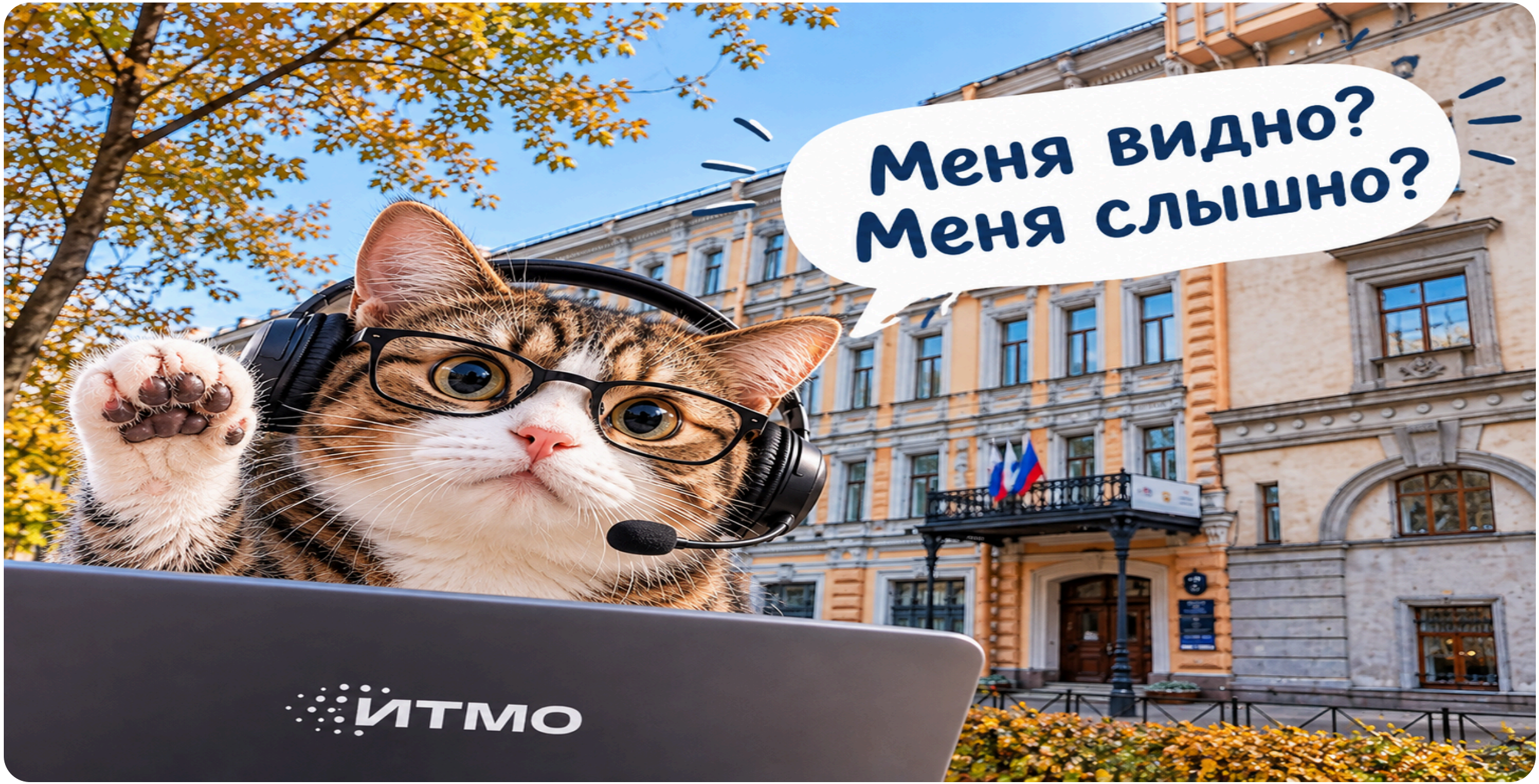
Семён Мартынов, Кирилл Дикалин, Георгий Семёнов

ИНСТИТУТ ПРИКЛАДНЫХ КОМПЬЮТЕРНЫХ НАУК

Национальный исследовательский университет ИТМО

12 ноября 2026

Проверка связи



Цели лекции

1. **Различать** хранилище, кэш, очередь задач и журнал событий по тому, что происходит с данными при записи и чтении, и **выбирать** модель под нагрузку.
2. **Оценивать** цену доступа к диску: последовательный и случайный ввод-вывод, IOPS и глубину очереди, `fsync`; **объяснять**, зачем нужны WAL и group commit.
3. **Сравнивать** B-дерево и LSM-дерево через усиление записи, чтения и пространства.
4. **Называть** гарантии и цену Redis (память, один поток команд, персистентность), RabbitMQ (exchange, подтверждения, DLQ) и Kafka (раздел, смещение, группа, репликация, retention).
5. **Считать** `prefetch`, число разделов и потребителей и **подбирать** подтверждения под допустимое окно потери данных.

Вопрос лекции:

что именно подтверждено, когда система ответила «OK», и сколько стоит это подтверждение?

План лекции

1. Производительность хранения: ввод-вывод, fsync, WAL, group commit
2. Индексные структуры: B+-дерево, LSM-дерево, усиления
3. Redis: данные в памяти, персистентность, горячие ключи
4. RabbitMQ: маршрутизация, подтверждения, очередь задач, Dead Letter Queue
5. Kafka: журнал, группы потребителей, репликация, retention
6. Сравнение моделей: кэш, очередь задач, журнал событий и выбор под нагрузку

Загадка

Брокер Kafka отвечает продюсеру «записано» (`acks=all`), хотя по умолчанию **не вызывает** **`fsync`** для сообщений. При этом документация утверждает, что подтверждённое сообщение не пропадает при отказе брокера. На тех же дисках линейная запись быстрее случайной **более чем в 6000 раз**.

Что значит «подтверждено», если на диск ничего не сброшено?
Где лежит подтверждённое сообщение и от каких отказов оно защищено?
Ингредиенты в 6.1 и лекции 4, разгадка в 6.5.

6.1 Где живут данные: порядки величин

Ступень	Задержка одного обращения	Как устроен обмен
Память (DRAM)	≈ 100 нс (лекция 4)	байты, адресация по месту
SSD (флэш)	50–150 мкс	запись страницами, стирание целыми блоками
HDD	1–10 мс	перемещение головки и ожидание оборота пластины
Сеть внутри ЦОД	≈ 0.5 мс (лекция 4)	запрос и ответ, RTT

- Между ступенями **порядки величин**: DRAM → SSD примерно в 10^3 раз, SSD → HDD ещё в 10–100 раз
- У хранилища нет «одной скорости»: **задержка** операции, **число операций** в секунду и **байты** в секунду зависят от размера и шаблона доступа
- Диск как ресурс описывают по USE (лекция 1): загрузка, насыщение (очередь к устройству), ошибки
- Не диск ускоряем, а **выбираем шаблон доступа**, который он делает дёшево: последовательный, крупными блоками, пачками (пакетирование, лекция 5)

Gregg. *Systems Performance* (2013), шкала задержек; Dean, *Latency Numbers Every Programmer Should Know* (~2012). Значения приблизительные.

6.1 Последовательный и случайный ввод-вывод



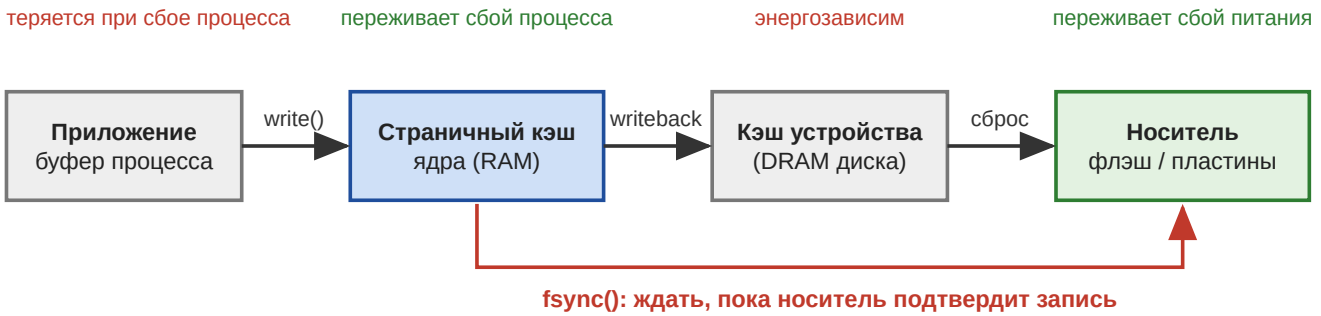
- HDD 7200 об/мин: 120 оборотов/с, оборот 8.33 мс, в среднем ждём **полоборота** (4.17 мс), плюс перемещение головки ≈ 10 мс (оценка из документации Kafka): порядка **70–100 IOPS** на диск
- Документация Kafka: массив из шести HDD (7200 об/мин) даёт ≈ 600 МБ/с на линейной записи и ≈ 100 кБ/с на случайной, **разрыв более чем в 6000 раз**. Jacobs (ACM Queue, 2009): последовательное чтение диска местами быстрее случайного доступа к памяти
- У SSD головки нет, и разрыв меньше, но не исчезает: ОС и устройство склеивают мелкие записи в крупные, флэш стирается блоками

6.1 IOPS, пропускная способность, задержка

Величина	Связь	Пример
Пропускная способность	IOPS × размер операции	$10^5 \times 4$ КБ ≈ 400 МБ/с
Глубина очереди L	закон Литтла $L = \lambda W$ (лекция 2)	10^5 IOPS при $W = 100$ мкс: $L = 10^5 \cdot 10^{-4} = 10$ операций «в полёте»
Предел одного синхронного клиента	$1/W$	глубина 1, $W = 100$ мкс: не больше 10 000 IOPS

- Один поток, ждущий каждую операцию, **не раскрывает** SSD: паспортные IOPS получают при глубокой очереди. Нужен параллелизм (асинхронный ввод-вывод, `io_uring`, лекция 10)
- **IOPS без размера и шаблона не имеют смысла** (лекция 1: «RPS без профиля»). Gregg: один и тот же диск при загрузке 99% давал 86 IOPS (случайные 8 КБ) и 21 284 IOPS (последовательные 512 Б)
- На узле смотрим `iostat -x`: задержку (`await`), длину очереди, загрузку `%util`. Для SSD `%util = 100%` ещё не насыщение: устройство обслуживает запросы параллельно. Рост очереди при $\rho \to 1$ это та же «хоккейная клюшка» (лекция 2)

6.1 Путь записи: страничный кэш и fsync



- `write()` возвращается после копирования в **страничный кэш** ядра. Сбой процесса данные не теряет, сбой ОС или питания теряет всё, что не сброшено
- `fsync()` блокирует вызывающего, пока устройство не подтвердит сброс: цена равна задержке носителя (на HDD миллисекунды). Это единственный способ узнать, что данные на носителе
- Кэш устройства обязан честно выполнять сброс, иначе `fsync` не защищает (у серверных SSD для этого есть защита от потери питания)
- Чтение идёт через тот же кэш: ОС отдаёт под него всю свободную память, и кэш остаётся «тёплым» после перезапуска процесса. Это нижний уровень кэша из лекции 5

6.1 WAL: сначала журнал



COMMIT OK возвращается после шага 2; шаг 3 может случиться через минуты

шаг 3 это контрольная точка; после сбоя проигрываем журнал с неё (redo)

- Правило WAL: изменение страницы данных пишут на диск **только после** того, как запись журнала об этом изменении сброшена на постоянное хранилище (PostgreSQL Documentation)
- Коммит стоит одного **последовательного** fsync журнала, а не записи всех изменённых страниц. Документация PostgreSQL: стоимость синхронизации журнала намного ниже стоимости сброса страниц данных
- Журнал нужен ещё для репликации (лекция 4) и восстановления на момент времени. Цена: изменение пишется дважды, а первое изменение страницы после контрольной точки может записать в журнал страницу целиком (защита от «разорванной» записи)

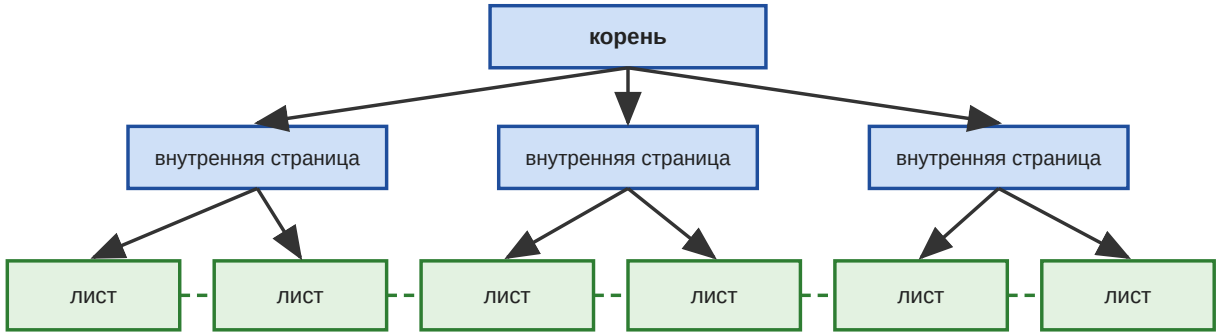
6.1 Group commit и цена надёжности

- Модель лекции 5: фиксированная цена a (fsync) и цена записи c. Пример: $a = 1$ мс, $c = 0.01$ мс. Один коммит на fsync: ≈ 990 коммитов/с. **50 коммитов** в общем fsync: $t = 1 + 50 \cdot 0.01 = 1.5$ мс, то есть $\approx 33\,000$ коммитов/с
- Group commit: пока идёт сброс, новые коммиты копятся и уходят **следующим** общим сбросом (PostgreSQL: один fsync может зафиксировать много транзакций). Одному последовательному клиенту это не помогает

Система	Режим	Что подтверждено при ответе	Окно потери при сбое ОС или питания
PostgreSQL	synchronous_commit = on / off	журнал сброшен / ещё нет	нет / до $3\times$ wal_writer_delay, порчи данных нет
MySQL InnoDB	innodb_flush_log_at_trx_commit = 1 / 2	fsync на коммите / запись в кэш ОС, fsync раз в секунду	нет / ≈ 1 с
Redis (AOF)	appendfsync always / everysec	fsync на команду / раз в секунду	нет / ≈ 1 с
Kafka	fsync отключён по умолчанию	запись в кэш ОС и репликация	зависит от реплик (6.5)
RabbitMQ	persistent + publisher confirms	подтверждение после записи на диск, сброс пачками	нет для подтверждённых

PostgreSQL Documentation, «Asynchronous Commit»; MySQL Reference Manual, innodb_flush_log_at_trx_commit; Redis Documentation, «Persistence»; RabbitMQ Documentation, «Consumer Acknowledgements and Publisher Confirms».

6.2 B+-дерево: поиск по страницам



листья хранят ключи и значения (или указатели на строки) и связаны в список: диапазон читается подряд

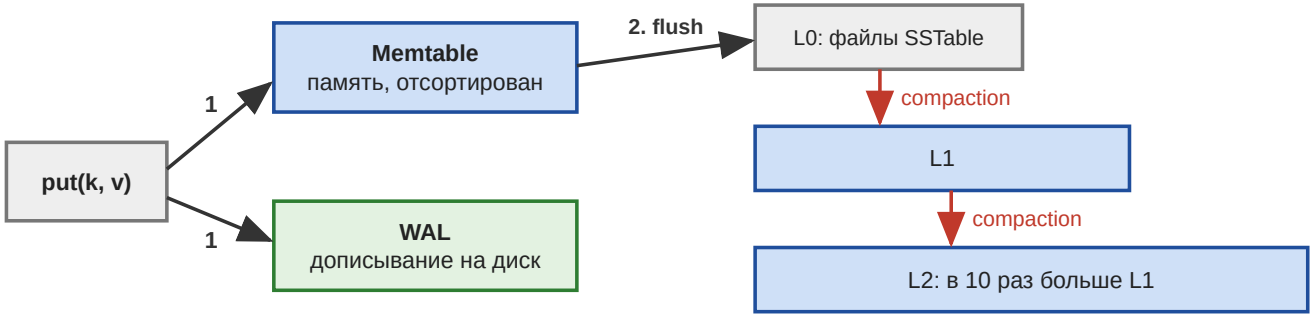
- Узел дерева это **страница** (PostgreSQL: 8 КБ, InnoDB: 16 КБ). Ветвление f измеряется сотнями, поэтому дерево низкое: $h = \lceil \log_f N \rceil$
- Пример: $N = 10^9$ ключей, $f = 500$: $\log_{500} 10^9 = 9 / \lg 500 \approx 3.3$, то есть $h = 4$. Верхние три уровня это ≈ 4000 страниц, около 32 МБ при 8 КБ: они живут в кэше (лекция 5), на диск уходит **≈ 1 чтение**
- Ключи отсортированы, поэтому дерево отвечает и на **диапазонные** запросы, чего не умеет хэш (лекция 5: схемы секционирования). Bayer, McCreight, 1972

6.2 B-дерево: цена записи

- Обновление **на месте**: страница читается, меняется и записывается целиком. Изменение 100 Б это запись страницы 8–16 КБ, то есть **усиление записи** ≈ 80–160 раз, и ещё запись в WAL (6.1)
- Ключ попадает в произвольное место: запись идёт в **случайные** страницы (6.1). Переполненная страница **расщепляется**: пишутся две страницы и родитель
- Возрастающие ключи (автоинкремент, время) дописывают правый край дерева и дёшевы. Случайные ключи (UUID v4) разбрасывают вставки по всему дереву: больше промахов кэша и расщеплений
- Чтение это сильная сторона: $\approx h$ страниц, из них 1–2 с диска. Фоновых слияний нет, поэтому **хвост задержки** предсказуем
- Все эти страницы нужно держать в буферном пуле: если рабочий набор не помещается, каждый запрос превращается в случайное чтение диска (6.1)

> B+-дерево платит за запись **сразу и случайно**, чтобы чтение было дешёвым и предсказуемым.

6.2 LSM-дерево: путь записи



каждый файл отсортирован и неизменяем; на диск пишем только последовательно

- **Запись**: дописывание в журнал (WAL, 6.1) и вставка в отсортированный **memtable** в памяти. Диск трогаем только последовательно
- Заполненный memtable сбрасывается в **SSTable**: неизменяемый отсортированный файл (в RocksDB размер задаёт `write_buffer_size`, по умолчанию 64 МБ). Файлы не правят, а **сливают** в фоне: *compaction*
- Уровни растут в T раз (RocksDB: `max_bytes_for_level_multiplier` = 10). Удаление это запись-**надгробие** (tombstone): место освободится при слиянии
- Идея: превратить случайные записи в последовательные ценой фоновой работы. O'Neil et al., 1996; термины memtable и SSTable из Bigtable (Chang et al., 2006)

6.2 LSM-дерево: чтение и compaction

- **Чтение**: memtable, затем файлы L0 (они перекрываются, смотрим все), затем по одному файлу на каждом уровне. Оценка RocksDB: число файлов L0 + число непустых уровней
- **Фильтр Блума** (Bloom, 1970) у каждого файла отвечает «точно нет» или «возможно есть»: большинство лишних чтений отсекается. Для диапазонных запросов фильтр не помогает

	Leveled (по уровням)	Tiered (size-tiered)
Идея	на уровне один упорядоченный ряд, при переполнении часть файлов сливается в нижний уровень	копим несколько серий одного размера, затем сливаем в одну большую
Усиление записи	высокое: на каждый уровень порядка T (RocksDB: «часто больше 10»)	ниже: на уровень порядка 1
Усиление чтения	низкое: один файл на уровень	выше: несколько серий на уровень
Место	меньше дубликатов	больше дубликатов до слияния

- Compaction **забирает** диск и CPU. Если запись быстрее слияния, файлов L0 всё больше, чтение дорожает, и движок **замедляет или останавливает запись** (write stall). Это очередь лекции 2: $\lambda_{\text{запись}} > \mu_{\text{compaction}}$

6.2 Три усиления

- **Усиление записи** W = байты, записанные на носитель / байты, записанные приложением. **Усиление чтения** R = обращения к страницам или файлам на запрос. **Усиление пространства** S = размер на диске / размер живых данных

	B+-дерево	LSM, leveled	LSM, tiered
W	страница целиком: ≈ 80–160 для записи 100 Б, плюс WAL	порядка T на уровень; пример из Tuning Guide RocksDB: $1 + 2 + 10 + 10 + 10 \approx 33$	ниже, порядка числа уровней
R	h страниц, 1–2 с диска	файлы L0 + число уровней, Bloom помогает	больше файлов
S	страницы заполнены частично	дубликаты и надгробия до слияния	больше дубликатов
Тип записи на диск	случайная	последовательная	последовательная

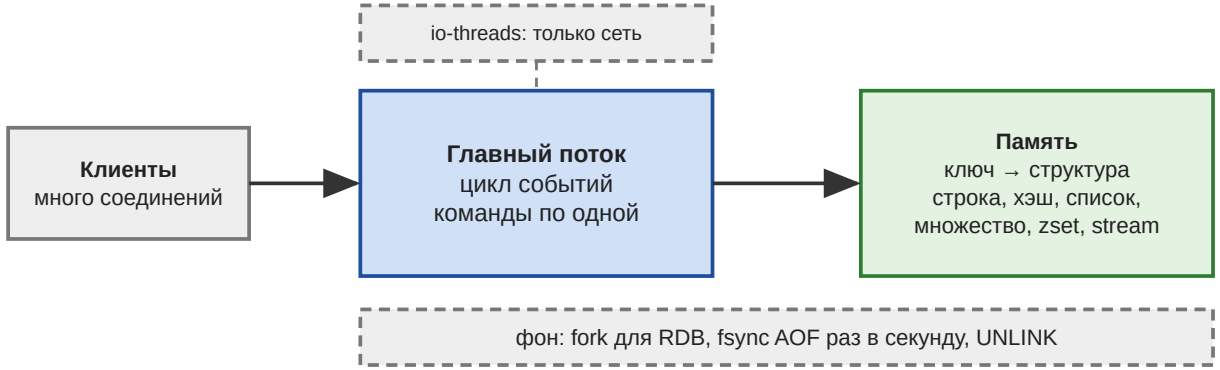
- Высокое W это ещё и **износ SSD**: ресурс флэша ограничен числом перезаписей (RocksDB Tuning Guide)
- **RUM-гипотеза** (Athanassoulis et al., 2016): нельзя одновременно минимизировать накладные расходы на чтение (**R**ead), обновление (**U**pdate) и память (**M**emory): улучшая два, ухудшаем третье

6.2 Что выбрать

Характер нагрузки	Склоняет к	Почему
Много чтений по ключу и диапазонам, умеренная запись, важен предсказуемый p99	B+-дерево (PostgreSQL, MySQL InnoDB)	чтение $\approx h$ страниц, нет фоновых слияний
Непрерывный поток записей (события, метрики, временные ряды)	LSM (RocksDB, LevelDB, Cassandra, HBase)	запись последовательная и пачками
Записей много, чтений мало, место важнее задержки чтения	LSM tiered	меньше W
Данные намного больше RAM, точечные чтения	LSM с фильтрами Блума	R сводится к одному-двум чтениям

- Выбор это функция **соотношения чтение/запись** и допустимого **хвоста**: фоновая compaction создаёт всплески p99 (лекция 1). Измеряйте p99 записи при **непрерывной** нагрузке, а не среднее на коротком тесте
- Мостик к 6.5: журнал Kafka вообще не индекс. Нет поиска по ключу и обновления на месте, есть «дописать» и «читать с смещения» (индекс по смещению разрежённый), поэтому почти всё время диск работает последовательно

6.3 Redis: модель и цена одного потока



- **Redis** это сервер структур данных в памяти: ключ → значение-структура (строка, хэш, список, множество, упорядоченное множество, поток). У каждой операции известная сложность
- Команды выполняет **один главный поток** по очереди: блокировок нет, каждая команда атомарна. С Redis 6.0 сетевой ввод-вывод можно вынести в потоки (`io-threads`), исполнение остаётся однопоточным. По FAQ узким местом чаще бывает память или сеть, а не CPU
- **Одна медленная команда блокирует всех** (head-of-line blocking): `KEYS *`, чтение огромной коллекции, долгий Lua-скрипт. Вместо них `SCAN`, `UNLINK` (удаление в фоне)
- Без pipelining клиент делает не больше $1/RTT$ команд/с на соединение: при $RTT = 0.5$ мс это ≤ 2000/с. Pipelining это пакетирование из лекции 5

6.3 Типичные сценарии

Сценарий	Структура и команды	Подвох
Кэш (cache-aside, лекция 5)	строки и хэши, `SET key val EX ttl`	вытеснение, stampede; кэш это часть ёмкости
Сессии, токены	строки или хэши с TTL	без персистентности сбой разлогинивает всех
Счётчики, лимиты частоты (лекция 7)	`INCR`, `EXPIRE`	атомарно только в пределах одного узла
Рейтинги, топ-N	sorted set: `ZADD`, `ZRANGE`, $O(\log N)$	один огромный ключ: и большой, и горячий
Очереди, уведомления	списки (`BLPOP`), Streams, Pub/Sub	Pub/Sub: доставка **не более одного раза**, офлайн-подписчик теряет сообщения
Блокировки	`SET lock token NX PX ttl`	аренда с TTL, см. «Репликация и блокировки»

Ключевой вопрос: **что случится, если данные пропадут?** Если их можно пересоздать, это кэш. Если нет, нужны персистентность и репликация, и надо знать их окно потери.

6.3 Персистентность: RDB и AOF

	RDB (снимок)	AOF (журнал команд)
Что	точечный снимок набора данных: `fork()` и запись дочерним процессом	каждая команда записи дописывается в журнал
Потеря при сбое	всё с момента последнего снимка (минуты)	`appendfsync always`: нет; `everysec` (по умолчанию): ≈ 1 с; `no`: решает ОС
Цена	`fork()` может остановить сервер на миллисекунды, на больших наборах до секунды	непрерывная запись; периодическое переписывание журнала (rewrite) тоже через `fork()`
Память	во время снимка копируются изменённые страницы (copy-on-write): запас **до** $2\times$	то же при rewrite
Рестарт	быстрый	медленнее: проигрывается хвост

- AOF это WAL из 6.1, а `everysec` это group commit по времени: `fsync` в фоновом потоке раз в секунду
- Redis рекомендует RDB + AOF вместе: снимки для бэкапов и быстрого рестарта. Для чистого кэша персистентность часто выключают
- Репликация тоже делает RDB-снимки, **даже если персистентность выключена** (кроме diskless-режима)

6.3 Память, большие и горячие ключи

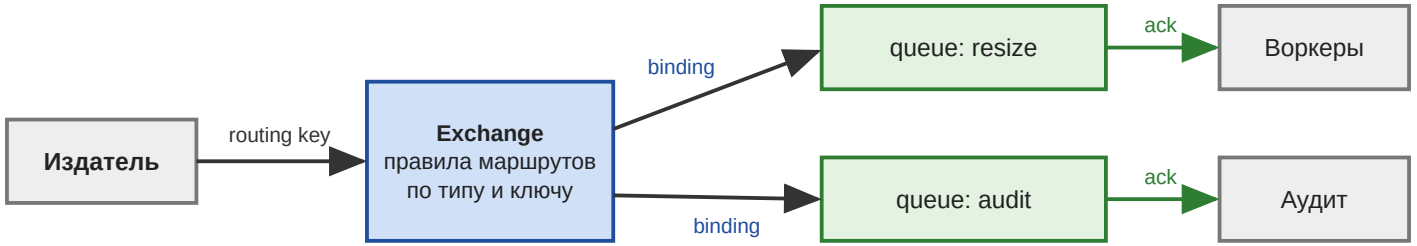
- **Всё должно поместиться в память**. `maxmemory` задают с запасом на накладные расходы и фрагментацию (по документации при 10 ГБ свободной памяти ставят 8–9), плюс запас до $2\times$ на `fork`. Политики вытеснения: лекция 5
- **Горячий ключ**: он лежит в одном слоте (16 384 слота, лекция 5), значит на одном первичном узле и в одном потоке команд. Кластер из n узлов не ускоряет один ключ (лекция 3). Меры:
  - локальный кэш в приложении с коротким TTL (лекция 5);
  - реплики для чтения (чтение может устареть, лекция 4);
  - разбиение ключа на k частей `key:0 … key:k-1`: запись и чтение дороже в k раз
- **Большой ключ** (список или хэш на миллионы элементов): команда над ним держит поток, `DEL` блокирует, а при миграции слота ключ переносится целиком. Ограничивайте рост: делите по времени или хэшу
- Redis Cluster допускает многоключевые команды только в **одном слоте**: ключи связывают хэш-тегами, `{user:42}:cart`, `{user:42}:profile`

6.3 Репликация, потери и блокировки

- Репликация **асинхронная** (лекция 4): первичный отвечает «OK», не дожидаясь реплик. Документация Redis Cluster: строгой согласованности нет, и при сбое первичного **подтверждённая запись может пропасть**. В терминах PACELC выбрана задержка
- `WAIT n timeout` блокирует клиента, пока n реплик не подтвердят запись: синхронность точечно. Но по документации Redis Cluster **не** даёт строгой согласованности даже с синхронной репликацией: реплика без записи может стать первичным
- Меньшая сторона разделения принимает записи до `NODE_TIMEOUT`, затем отказывает; записи этого окна могут пропасть при failover (лекция 4: split-brain, потеря хвоста)
- **Блокировка** `SET lock token NX PX ttl` это **аренда**: пауза клиента (GC, лекция 10) дольше TTL, и владельцев двое. Kleppmann (2016): для **эффективности** такой замок допустим, для **корректности** нужен fencing token (лекция 4), которого у Redlock нет. Sanfilippo возражал в «Is Redlock safe?»

> Redis как **база данных** это осознанный выбор: персистентность, реплики и допустимое окно потери.
>
> Redis как **кэш**: потеря допустима, источник истины в другом месте.

6.4 RabbitMQ: exchange, binding, queue



сообщение копируется в каждую подходящую очередь и лежит там до подтверждения (ack)

- Издатель публикует в **exchange**, а не в очередь. Exchange по **типу** и **ключу маршрутизации** решает, в какие очереди положить копию, через **bindings**. Типы: `direct` (точный ключ), `fanout` (во все), `topic` (шаблоны `orders.*.created`), `headers`; безымянный exchange направляет по имени очереди
- Несколько очередей это **несколько копий** (pub/sub). Несколько потребителей одной очереди это **разделение работы** (competing consumers): сообщение получает один
- Издатель не знает потребителей: маршруты меняют без его правки. **Брокер ведёт состояние доставки каждого сообщения**
- Очередь выполняется **одним ядром** узла: для параллелизма нужно несколько очередей (документация RabbitMQ)

6.4 Подтверждения: три границы надёжности

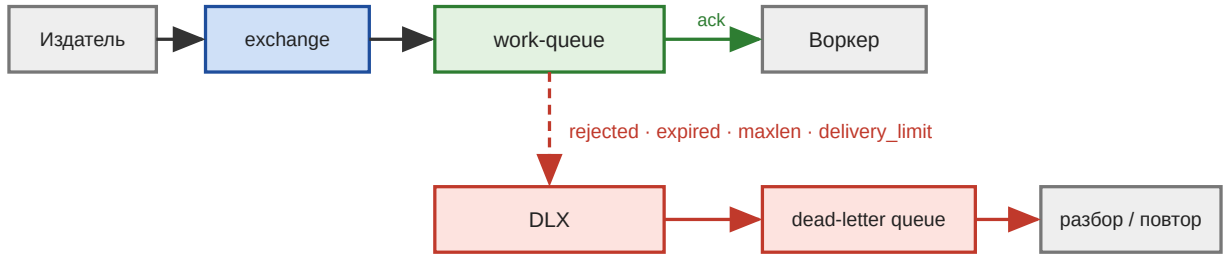
Участок	Механизм	Что гарантирует	Без него
Издатель → брокер	**publisher confirms**	брокер принял; для persistent в durable-очереди подтверждение **после записи на диск**; для quorum-очереди после принятия кворумом реплик	издатель не узнает о потере
Хранение в очереди	durable-очередь + persistent-сообщение (`delivery_mode=2`)	сообщение переживает рестарт узла	пропадает при рестарте
Очередь → потребитель	**ack** (ручной режим)	сообщение удаляется только после `ack`; при обрыве канала возвращается в очередь	auto-ack: потеря при сбое потребителя

- Итог: **«хотя бы раз»** (лекции 4 и 5). После сбоя потребителя сообщение придёт снова, поэтому обработчик **идемпотентный**; флаг `redelivered` подсказывает, но не заменяет ключ идемпотентности
- Для persistent-сообщений подтверждение издателю идёт после сброса на диск **пачкой**: при постоянной нагрузке до сотен миллисекунд. Обрабатывайте подтверждения асинхронно и публикуйте пачками
- Транзакции AMQP снижают пропускную способность примерно в 250 раз (документация): используйте confirms

6.4 Очередь задач: prefetch и воркеры

- prefetch p (basic.qos): сколько **неподтверждённых** сообщений может быть «в полёте» на канале. Пока нет ack , брокер новых не шлёт. Это закон Литтла (лекция 2): в полёте не больше $p \cdot m$ сообщений для m воркеров
- Пример: обработка $t = 10$ мс, доставка и ack занимают $RTT = 2$ мс. При $p = 1$ воркер ждёт сеть между сообщениями: $1/(t + RTT) \approx 83$/с вместо $1/t = 100$/с. При $p \geq 2$ следующее сообщение уже у воркера
- Для **долгих** задач (секунды и минуты, как обработка изображений на семинаре) берут малое p: иначе медленный воркер копит p сообщений, пока остальные простаивают (как round robin против least connections, лекция 5). Для коротких документация называет оптимальными значения 100–300
- Метрики: глубина очереди, **возраст старейшего**, число неподтверждённых, «consumer capacity» (подсказка добавить воркеров). Прирост воркеров повышает μ (лекция 2)
- Тайм-аут подтверждения по умолчанию **30 минут**: воркер, не ответивший вовремя, теряет канал, сообщения возвращаются
- **Порядок**: FIFO только для одного потребителя. С несколькими потребителями и повторной доставкой порядок нарушается; для строгого порядка есть single active consumer

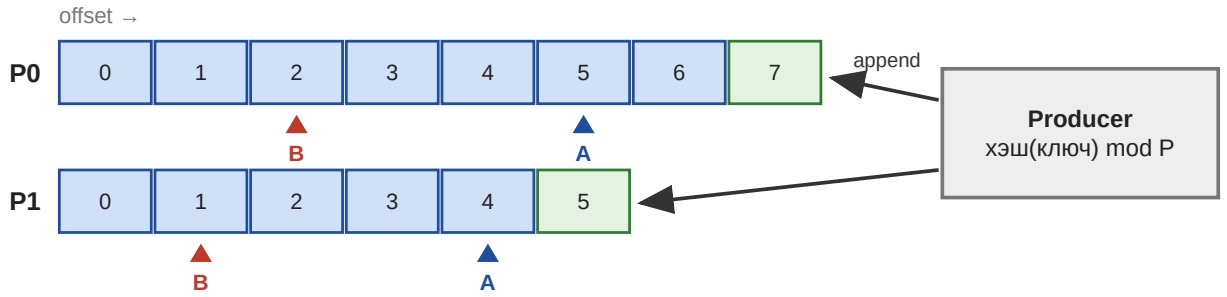
6.4 Dead Letter Queue и отравленные сообщения



заголовок x-death хранит причину, очередь и счётчик; без DLX такие сообщения теряются

- Сообщение «умирает» по четырём причинам: **rejected** (`nack` или `reject` с `requeue=false`), **expired** (TTL), **maxlen** (очередь переполнена), **delivery_limit** (quorum-очередь: превышено число повторных доставок)
- **Отравленное сообщение** падает при каждой обработке и без лимита крутится по кругу. В RabbitMQ 4.0 у quorum-очередей лимит доставок **по умолчанию 20** (раньше не было). После лимита сообщение **отбрасывается**, если не задан DLX
- **Отложенный повтор**: DLQ с TTL и без потребителей, чей DLX ведёт обратно в рабочий exchange. Сообщение вернётся через TTL (backoff и jitter, лекция 7)
- DLQ нужен **владелец и алерт**: рост DLQ это ошибка в коде или данных. Quorum-очереди умеют `at-least-once` dead-lettering (по умолчанию `at-most-once`)

6.5 Kafka: topic, partition, offset



A и B: две независимые группы потребителей, у каждой свой offset; чтение ничего не удаляет

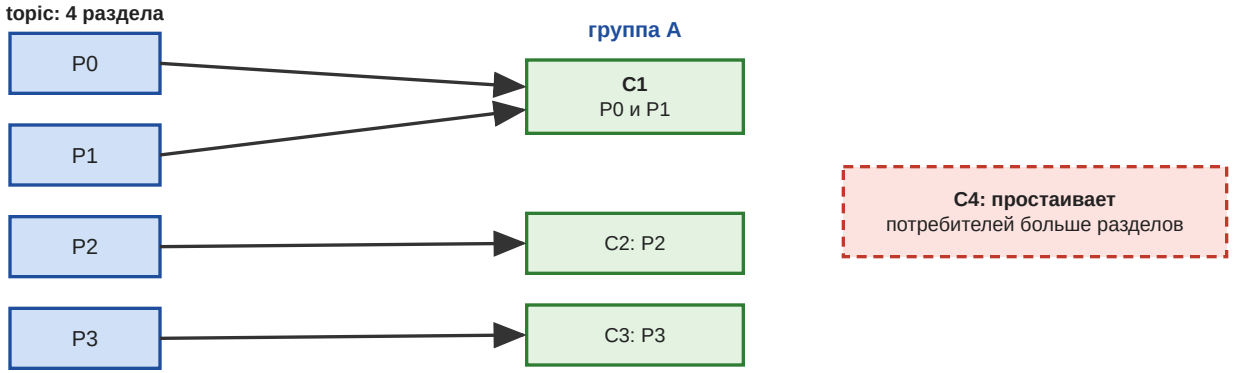
- **Topic** это именованный поток, поделённый на **разделы** (partitions). Раздел это **журнал только для дописывания**; каждой записи в нём присвоено **смещение** (offset), порядковый номер
- Порядок гарантирован **только внутри раздела**. Ключ определяет раздел: хэш ключа по модулю числа разделов (лекция 5)
- Чтение **не удаляет**: потребитель хранит **свой** offset, поэтому читатели независимы и могут читать заново. Документация: дизайн «ближе к журналу базы данных, чем к традиционной системе обмена сообщениями»
- Раздел это единица **порядка, параллелизма, репликации и хранения**

6.5 Почему Kafka быстра: 6.1 в действии

Приём	Механизм из 6.1	Эффект
Дописывание в конец файла сегмента	последовательный ввод-вывод	линейная запись ≈ 600 МБ/с против ≈ 100 кБ/с случайной (документация)
Страничный кэш ОС вместо кэша в куче JVM	write-behind и read-ahead	нет двойного кэширования и GC-пауз, кэш тёплый после рестарта
Пачки (`batch.size` , `linger.ms` , лекция 5) и сжатие	амортизация фиксированной цены	меньше мелких операций диска и сети
`sendfile`	из страничного кэша прямо в сокет	нет копирования через память приложения
Разрежённый индекс по offset	нет дерева, нет записи на месте	все операции $O(1)$

- Kafka **отказалась** от того, что делает индексы дорогими: нет поиска по ключу, обновления на месте, удаления отдельной записи
- Цена: читать выгодно **подряд**. Потребитель, отставший на сутки, читает с диска, а не из кэша

6.5 Consumer group



- В группе раздел читает **один** потребитель: полезных потребителей не больше числа разделов. Группы независимы: группа это «приложение»
- Прогресс хранится как **committed offset** в служебном топике `__consumer_offsets`. **Lag** = конец журнала − committed offset: глубина очереди Kafka, ожидание $W \approx \mathrm{lag}/\mu$ (лекция 5)
- **Размер**: 20 000 сообщений/с, обработка 5 мс: $L = \lambda W = 20\,000 \cdot 0.005 = 100$ потребителей; при загрузке 70% (ρ, лекция 3) ≈ 143 раздела
- Число разделов выбирают **заранее**: уменьшить нельзя, а увеличение меняет `хэш(ключ) mod P`, и ключи переезжают в другие разделы
- **Ребалансировка** при входе или выходе потребителя. Новый протокол (KIP-848, GA в Kafka 4.0) назначает разделы на брокере и делает перераспределение инкрементальным

6.5 Репликация, ISR и подтверждения



acks	Продюсер получает ответ после	Теряем при отказе лидера
0	отправки	не узнаем о потерях
1	записи лидером	записи, не дошедшие до фолловеров
all	записи **всеми репликами ISR** (не меньше min.insync.replicas)	ничего, пока в ISR есть живая реплика

- acks=all с Kafka 3.0 по умолчанию, но min.insync.replicas равен 1: при ISR из одного лидера all ведёт себя как 1. Типовая схема: replication.factor=3, min.insync.replicas=2. Лидером не станет реплика вне ISR (unclean.leader.election.enable=false): выбор согласованности вместо доступности (CAP)

> **Разгадка.** «Подтверждено» значит: запись есть в кэше ОС **всех** реплик ISR. Отказ одного брокера данные не теряет, они есть у других. Не защищает от **одновременного** отказа всех реплик (питание стойки): реплики держат в разных стойках и зонах (лекция 3).

6.5 Retention, compaction и повторное чтение



- Запись **не удаляется при чтении**: хранится `retention.ms` (по умолчанию 7 суток) и/или `retention.bytes` (по умолчанию без ограничения). Удаляются **целые закрытые сегменты** (`segment.bytes` = 1 ГиБ), поэтому место освобождается порциями
- **Повторное чтение**: потребитель может сдвинуть offset назад: пересчитать агрегат после ошибки, подключить нового потребителя, восстановить кэш или индекс. В очереди (6.4) подтверждённое сообщение исчезло
- **Log compaction** (`cleanup.policy=compact`): остаётся минимум **последнее значение каждого ключа**, удаление ключа это запись с пустым значением. Журнал становится таблицей «ключ → последнее значение»
- **Tiered storage** (KIP-405, production-ready в 3.9): старые сегменты уходят в объектное хранилище, на брокере остаётся горячая часть
- Ёмкость: $\text{диск} = \text{скорость записи} \cdot T_{\text{retention}} \cdot RF$. Пример: 20 МБ/с за 7 суток при $RF = 3$: $20 \cdot 604\,800 \cdot 3 \approx 36$ ТБ

6.5 Гарантии доставки и подводные камни

Семантика	Как получить	Цена
не более одного раза	commit offset **до** обработки	потеря при сбое
хотя бы раз	commit offset **после** обработки	дубли: идемпотентный обработчик (лекция 5)
ровно один раз (внутри Kafka)	идемпотентный продюсер, транзакции, `read_committed`	пропускная способность и задержка; внешние системы (БД, HTTP) вне гарантии

- Идемпотентный продюсер включён по умолчанию с Kafka 3.0: повторные отправки не создают дублей в разделе
- **Отравленное сообщение блокирует раздел**: потребитель не может сдвинуть offset. В обычных группах нет подтверждения каждой записи и DLQ, их строят в приложении (отдельный топик ошибок). Для share groups DLQ добавлен в Kafka 4.4 (KIP-1191)
- **Медленный потребитель** задерживает свой раздел, остальные идут дальше. **Перекос ключей** даёт горячий раздел (лекции 3 и 5)
- **Слишком много разделов**: открытые файлы и области отображения памяти (`vm.max_map_count`), время восстановления и ребалансировки

6.6 Четыре модели: что происходит с данными

	Хранилище (БД)	Кэш (Redis)	Очередь задач (RabbitMQ)	Журнал событий (Kafka)
Роль данных	**источник истины**	**копия**, можно пересоздать	**работа**, которую надо выполнить	**факты** за срок хранения
Что делает чтение	ничего	ничего	**удаляет** после `ack`	ничего, у читателя свой offset
Перечитать	да	пока не вытеснено	нет	да, в пределах retention
Порядок	транзакции, ключи	не нужен	FIFO в очереди, ломается при redelivery	внутри раздела
Единица масштаба	секция, шард	слот, шард	очередь (одно ядро), воркеры	раздел
Что ломается	дорогая запись (6.1–6.2)	потеря, вытеснение, stampede	рост очереди, отравленные сообщения	перекос ключей, lag, retention
Следить за	p99, lag реплик	hit ratio, память	глубина, **возраст старейшего**, unacked	consumer lag, ISR

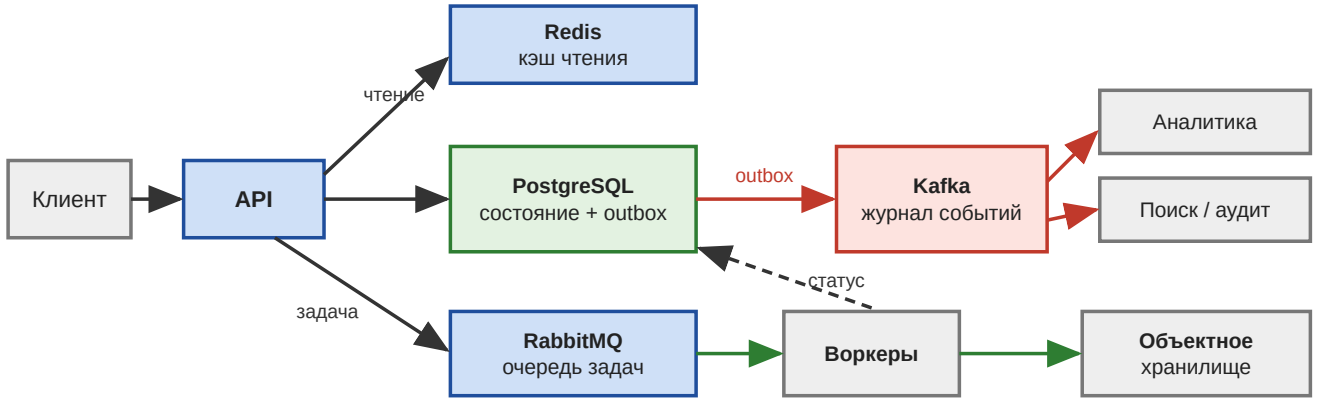
> Очередь хранит **работу**, и после выполнения работа исчезает. Журнал хранит **факты**, и сколько раз их читать, решает читатель.

6.6 Выбор по характеру нагрузки

Характер нагрузки или вопрос	Модель и инструмент	Цена
Нужны поиск по ключу и условию, транзакции, долговечность	хранилище: PostgreSQL и аналоги (6.1–6.2)	запись и индексы
Чтения повторяются, источник насыщен, данные можно пересоздать	кэш: Redis, cache-aside (5.2)	устаревание, stampede, холодный старт
Тяжёлая операция вынесена из запроса, каждую задачу выполняет **один** воркер	очередь задач: RabbitMQ	`ack`, идемпотентность, DLQ
Один факт нужен **нескольким** независимым потребителям, нужен повтор, поток большой	журнал событий: Kafka	порядок только в разделе, lag, retention
Задачи и повтор или несколько читателей вместе	гибрид: Kafka share groups, RabbitMQ Streams (границы размываются)	новая эксплуатация, меньше гарантий порядка
Нагрузка мала, СУБД уже есть	таблица в БД: `SELECT … FOR UPDATE SKIP LOCKED`	не масштабируется как брокер, нагрузка на БД

Нет измеренного ограничения: нет оснований для нового компонента (лекция 5, анти-паттерны).

6.6 Как сочетают модели



- Типовая связка: **БД это источник истины**, Redis ускоряет чтение, RabbitMQ раздаёт задачи, Kafka рассылает **факты** остальным системам. Из БД в Kafka события идут через **transactional outbox** (лекция 5), чтобы запись и публикация не разошлись
- Сквозной проект и семинар 6: API → очередь → воркер → результат; клиент получает идентификатор, состояние лежит в БД (лекция 5)
- **Границы размываются.** Kafka 4.2 (февраль 2026): share groups (KIP-932), то есть подтверждение каждой записи без порядка и без привязки числа потребителей к числу разделов. В RabbitMQ есть Streams: журнал с повторным чтением. Выбираем по **модели данных и гарантиям**, а не по названию

Чек-лист инженера

1. Для каждого шага ответьте: **что подтверждено** при «OK», где лежит и какое **окно потери** допустимо. Под него выбирайте `fsync`, репликацию, `acks` и confirms
2. Диск: считайте IOPS × размер и глубину очереди $L = \lambda W$; группируйте записи (group commit, пачки); смотрите `iostat -x`
3. Индекс: преобладают чтения и важен p99 — B+-дерево; поток записей — LSM, и следите за W и всплесками при compaction
4. Redis: ответьте «что если данные пропадут?»; `maxmemory` с запасом на `fork`; без `KEYS`; горячий ключ — локальный кэш или разбиение
5. RabbitMQ: confirms + `ack` + **идемпотентный** обработчик; `prefetch` по RTT и длине задачи; DLX и лимит повторов; метрика — возраст старейшего
6. Kafka: ключ определяет порядок; число разделов $\geq \lambda W / \rho$; `acks=all`, $RF = 3$, `min.insync.replicas=2`; следите за lag и ISR
7. Не добавляйте компонент без измеренного ограничения (лекция 5): записывайте, что он добавляет, как измеряем, как откатываем

Заключение

Тема	Главное
Хранение	случайная операция HDD ≈ 10 мс; линейная запись быстрее случайной до 6000× (Kafka); IOPS × размер; $L = \lambda W$
fsync и WAL	write кладёт в кэш ОС, fsync на носитель; WAL это последовательный журнал; group commit делит fsync на b коммитов
B+-дерево	$h = \lceil \log_f N \rceil$ (4 при $N = 10^9$, $f = 500$); запись = страница целиком
LSM-дерево	memtable → SSTable → compaction; $W \sim T\times$ уровни; чтение спасают фильтры Блума
Redis	один поток команд; RDB и AOF; асинхронная репликация; горячие и большие ключи
RabbitMQ	exchange → binding → queue; confirms + ack = «хотя бы раз»; prefetch; DLX
Kafka	раздел = журнал, offset, группа; acks=all + ISR; retention и повторное чтение
Выбор	кэш хранит копию, очередь хранит работу, журнал хранит факты

Семинар 6: RabbitMQ, асинхронный конвейер обработки изображений и обзор Kafka. Лекция 7: **устойчивость и управление нагрузкой** (перегрузка очередей, лимиты, повторы, circuit breaker).

Список источников (1/2)

- Apache Kafka Documentation. *Design* (Persistence, Efficiency, Replication, Message Delivery Semantics); *Hardware and OS*; *Topic-Level Configs*; *Upgrading* (4.0: только KRaft, KIP-848 GA; 4.2: share groups, KIP-932)
- Apache Kafka. *Apache Kafka 3.9.0 Release Announcement* (tiered storage); Confluent. *Introducing Apache Kafka 4.4.0* (22 октября 2026, KIP-1191)
- Kreps, Narkhede, Rao. *Kafka: a Distributed Messaging System for Log Processing* (NetDB 2011)
- Jacobs. *The Pathologies of Big Data* (ACM Queue, 2009)
- RabbitMQ Documentation (4.2). *Consumer Acknowledgements and Publisher Confirms*; *Consumers*; *Quorum Queues*; *Dead Letter Exchanges*; release notes 4.0 (удалено зеркалирование классических очередей, `delivery-limit` = 20)
- Redis Documentation. *Persistence*; *Scale with Redis Cluster*; *Redis Cluster Specification*; *Administration*; *FAQ*; *Using pipelining to speed up Redis queries*
- Kleppmann. *How to do distributed locking* (2016); Sanfilippo. *Is Redlock safe?* (2016)

Список источников (2/2)

- PostgreSQL Documentation. *Write-Ahead Logging (WAL)*; *Asynchronous Commit*; *SELECT* (блокировки строк, `SKIP LOCKED`)
- MySQL Reference Manual. `innodb_flush_log_at_trx_commit`; *InnoDB Page Size*
- Gregg. *Systems Performance* (Addison-Wesley, 2013); *Performance Visualizations* (USENIX LISA 2010); Dean. *Latency Numbers Every Programmer Should Know* (~2012)
- O'Neil, Cheng, Gawlick, O'Neil. *The Log-Structured Merge-Tree (LSM-Tree)* (Acta Informatica 33(4), 1996)
- Bayer, McCreight. *Organization and Maintenance of Large Ordered Indexes* (Acta Informatica 1(3), 1972)
- Chang et al. *Bigtable: A Distributed Storage System for Structured Data* (OSDI 2006)
- Athanassoulis et al. *Designing Access Methods: The RUM Conjecture* (EDBT 2016); Bloom. *Space/Time Trade-offs in Hash Coding with Allowable Errors* (CACM 13(7), 1970)
- RocksDB Wiki. *Leveled Compaction*; *Compaction*; *RocksDB Tuning Guide*
- Kleppmann. *Designing Data-Intensive Applications* (O'Reilly, 2017): гл. 3 «Хранение и извлечение», гл. 11 «Потоковая обработка»

Обратная связь

Анонимная форма